

Notify

Overview

While working on Kyn, we were exploring ways to solve one of the biggest problems in the venue booking ecosystem—**inventory synchronization**.

Unlike industries such as airlines or cinemas, sports turfs and activity centers don't have a shared inventory protocol. Most venue owners list the same slot across multiple booking platforms, but these platforms don't expose APIs that allow inventory to stay synchronized.

Our long-term vision was to eventually build something similar to ONDC for slot-based venues—a common inventory layer that different booking platforms could connect to. However, building such an ecosystem would require participation from large industry players, making it a long-term initiative rather than an immediate solution.

As a short-term experiment, I was asked to explore whether we could automate inventory updates without relying on APIs.

Within **two days**, I built a working proof of concept that used Android notifications as the integration layer.

Although it was only a demo project, it successfully validated that notifications could be transformed into real-time inventory events.

Background

Whenever a booking happens on platforms like TurfTown or District, venue managers immediately receive a confirmation notification on their phones.

That made me wonder:

"If venue managers already receive booking information through notifications, do we actually need APIs?"

Instead of integrating with booking platforms, maybe we could integrate with the notifications they already send.

That became the entire idea behind this MVP.

Why We Didn't Build APIs

Our original vision was much bigger.

We wanted to build a unified inventory protocol for slot-based venues, similar to how cinema chains maintain seat availability across different ticketing platforms.

But that kind of solution only works when multiple large players agree to participate.

Since no common protocol existed—and competitors had no incentive to expose their APIs—we needed another way to validate the idea.

This notification-based solution was built to answer one simple question:

Can we automate inventory synchronization without requiring any cooperation from other platforms?

The Business Problem

One of our venue partners, **VGP Turf Arena**, listed their inventory on multiple booking platforms including TurfTown and District.

If someone booked a slot on one platform, the venue staff had to manually block the same slot inside Kyn.

Every missed update increased the risk of double bookings.

The existing workflow looked like this:

Booking happens

↓

Venue receives notification

↓

Staff opens Kyn dashboard

↓

Searches for slot

↓

Blocks inventory manually

↓

Repeats several times every day

Although simple, this process depended entirely on humans remembering to perform each step.

My Goal

Build a lightweight Android application capable of:

- Reading booking notifications
- Understanding booking details
- Identifying venue, date and slot
- Calling Kyn APIs automatically
- Closing inventory without manual intervention

The objective wasn't to build a polished product.

The objective was to validate whether notifications could become an integration layer.

Tech Stack

Since this was an exploration project, I optimized for speed instead of perfection.

I built the entire MVP using:

Mobile

- Kotlin
- Android Studio
- Android Notification Listener Service

Backend

- Supabase
- REST APIs

Deployment

- Vercel

Admin Dashboard

- React

The entire project was completed in approximately **two days**, using AI-assisted development with Claude to rapidly prototype the architecture and implementation.

How It Worked

The onboarding process was intentionally simple.

Once the APK was installed, the venue manager granted Notification Access permission on Android.

Inside the application, they selected which applications should be monitored.

For the demo, I configured:

- WhatsApp
- Gmail

I also manually registered the mobile number and email address whose notifications should be parsed.

This ensured that only booking confirmations from trusted sources were processed.

The Demo Flow

Once setup was complete, the workflow looked like this.

A booking confirmation was sent through WhatsApp or Gmail containing details such as:

- Venue
- Date
- Time
- Booking confirmation

↓

The application intercepted the notification.

↓

The parsing engine extracted the booking details.

↓

Structured booking data was generated.

↓

A backend API was triggered.

↓

The corresponding slot was automatically marked unavailable inside the Kyn portal.

Without anyone opening the dashboard, inventory was updated within seconds.

Watching the slot disappear automatically after receiving a notification became the most satisfying part of the demo.

Building the Parser

The notifications weren't designed for machines.

They were written for humans.

That meant every message had slightly different wording.

Instead of relying on AI, I built a deterministic parser that looked for predefined booking patterns.

From every notification, I extracted:

- Venue
- Booking date
- Slot timing
- Booking status

Once validated, the parser converted the notification into a structured API payload.

Challenges

The biggest challenge wasn't writing Kotlin code.

It was figuring out how to reliably extract structured information from completely unstructured notification text.

Since every platform formats confirmations differently, the parser had to remain flexible while still being predictable.

Another challenge was ensuring the application only processed booking notifications and ignored everything else.

This required app-level filtering as well as sender-level validation.

Pitching the Idea

Once the prototype was ready, we demonstrated it to one of our organizer partners.

Initially, we believed this could become an add-on feature priced around **₹500 per month**.

The discussion took an interesting turn.

The organizer explained that whenever double bookings happened, they usually handled it manually by either:

- Offering another slot
- Giving customers a 50% discount
- Refunding part of the booking

Although the existing process worked, it consumed operational time and affected customer experience.

When we demonstrated automatic inventory synchronization, they immediately saw its value.

Surprisingly, they said they would be comfortable paying around **₹5,000 per month** for a reliable solution because preventing even a few double bookings every month would save significantly more than that.

That conversation completely changed how we valued the product.

Key Learnings

This project taught me that validating a business idea doesn't always require months of development.

Sometimes, a small but functional prototype is enough to start meaningful conversations with customers.

Technically, I learned how Android's Notification Listener framework can be used beyond its traditional purpose and how event-driven architectures can be created without direct platform integrations.

From a product perspective, the biggest lesson was understanding the difference between solving a technical problem and solving an operational one.

The organizer wasn't buying a notification parser.

They were buying peace of mind.

What's Next?

Although this MVP achieved its objective, it was never intended to be the final solution.

The long-term vision remains much bigger.

Instead of relying on notifications, the goal is to build a dedicated inventory layer for slot-based venues that can integrate directly with multiple booking platforms—similar to how centralized inventory systems operate in industries like cinema and hospitality.

This prototype simply proved that the business problem was worth solving and that there was genuine willingness to pay for a better solution.

Despite being completed in just two days, the project successfully validated both the technical feasibility and the commercial potential of notification-driven inventory synchronization.